

TDK POS 웹 프로젝트 초기 설치 및 설정 기록

01_TDK_Web / tdk - pos - Next.js + TypeScript 기반 POS 개발 시작 문서

이 문서는 지금까지 실제로 선택한 프로젝트 생성 옵션과 설치 과정을 나중에 다시 확인할 수 있도록 정리한 기록이다. 데이터베이스는 MariaDB의 TDK_POS, 웹 프로젝트 폴더명은 tdk - pos로 구분한다.

항목	선택/상태	용도
Node.js	v22.17.0	Next.js 실행 환경
프로젝트 상위 폴더	01_TDK_Web	문서와 실제 앱을 함께 보관
웹 프로젝트명	tdk - pos	실제 Next.js 애플리케이션
DB 이름	TDK_POS	MariaDB 데이터베이스
TypeScript	Yes	타입 검사를 통한 개발 안정성
Linter	ESLint	코드 오류/품질 검사
React Compiler	No	초기 구성에서는 사용하지 않음
Tailwind CSS	Yes	UI 및 반응형 화면 스타일링
src/ directory	Yes	애플리케이션 소스 분리
App Router	Yes	Next.js 권장 라우팅 구조
Import alias	기본값 @/*	긴 상대 경로를 줄임
AGENTS.md	Yes	AI 코딩 도구용 프로젝트 지침

1. 프로젝트를 시작하기 전 상태

기존에 시도하다 중단한 pos_order_db는 삭제하고, 새 POS용 데이터베이스를 TDK_POS라는 이름으로 새로 생성했다. 설계 SQL의 데이터베이스명도 TDK_POS로 변경하여 phpMyAdmin에서 실행했다.

참고자료 보관 구조

프로젝트 상위 폴더에는 데이터베이스 설계 가이드 PDF와 MariaDB SQL을 docs 폴더에 보관하고, 실제 Next.js 프로그램은 tdk-pos 하위 폴더에 생성하는 구조를 사용한다.

```
01_TDK_Web/  
  docs/  
    TDK_POS_database_design_guide_ko.pdf  
    TDK_POS_mariadb_10_11.sql  
  tdk-pos/ ← ■■ ■ ■■■■
```

2. Node.js 확인

VS Code의 터미널은 Windows cmd를 사용했다. PowerShell이 아니어도 Next.js 개발에는 문제가 없다. 프로젝트 생성 전에 다음 명령으로 Node.js 설치 상태를 확인했다.

```
node -v
```

```
확인된 버전: v22.17.0
```

3. Next.js 프로젝트 생성

VS Code 터미널에서 다음 명령을 실행했다.

```
npx create-next-app@latest
```

실행 과정에서 create-next-app@16.3.5 설치가 필요하다는 메시지가 표시되었고, Ok to proceed? (y) 질문에 y를 입력하여 진행했다.

프로젝트 이름

웹 프로젝트 이름은 tdk-pos로 지정했다. 데이터베이스 이름 TDK_POS와 웹 프로젝트 이름은 서로 별개의 이름이다.

```
■■ ■■:  
D:\10_■■■■■■_■■\01_TDK_Web\tdk-pos
```

4. create - next - app에서 선택한 옵션

Recommended defaults를 그대로 사용하지 않고 No, customize settings를 선택하여 아래 옵션을 직접 지정했다.

TypeScript - Yes

프로그램 언어로 TypeScript를 사용한다. 주문, 결제, 재고처럼 데이터 정확성이 중요한 POS에서 타입 오류를 개발 단계에서 발견하는 데 유리하다.

Lint - ESLint

코드를 작성하면서 오류 가능성이나 좋지 않은 코드 패턴을 검사한다.

React Compiler - No

초기 프로젝트에서는 사용하지 않는다. POS 핵심 기능 구현에 필수적인 기능이 아니므로 구성을 단순하게 유지한다.

Tailwind CSS - Yes

POS 버튼, 메뉴 카드, 테이블 화면과 PC/태블릿/스마트폰 반응형 레이아웃을 작성하는 데 사용한다.

src/ directory - Yes

실제 애플리케이션 코드를 src 아래에 모아 프로젝트 루트를 깔끔하게 유지한다.

App Router - Yes

Next.js의 App Router 구조를 사용한다. /pos, /kitchen, /qr, /admin 같은 경로를 구성하기에 적합하다.

Import alias - No (기본 @/* 유지)

별도 별칭을 만들지 않고 기본 @/*를 사용한다. 예: import { db } from "@db"

AGENTS.md - Yes

ChatGPT나 VS Code의 AI 코딩 도구가 프로젝트를 수정할 때 최신 Next.js 코딩 지침을 참고할 수 있도록 포함한다.

5. 설치 완료 상태

create - next - app 실행 결과 364개 패키지가 추가되고 365개 패키지가 감사(audit)되었으며, 설치 당시 0 vulnerabilities로 표시되었다. 마지막으로 다음 성공 메시지가 확인되었다.

```
Success! Created tdk-pos at
D:\10_■■■■■■■■■■\01_TDK_Web\tdk-pos
```

npm의 새 버전 알림도 표시되었지만 프로젝트 생성과는 무관하므로 현재 단계에서는 npm을 별도로 업데이트하지 않았다.

6. 생성 후 예상 프로젝트 구조

```
01_TDK_Web/
  docs/
    TDK_POS_database_design_guide_ko.pdf
    TDK_POS_mariadb_10_11.sql
  tdk-pos/
```

```
src/  
  app/  
public/  
package.json  
tsconfig.json  
AGENTS.md  
...
```

7. 앞으로 사용할 기술 구성

현재까지 확인한 전체 구성은 아래와 같다. Drizzle은 아직 이 문서 작성 시점에는 설치 전이며, 다음 단계에서 MariaDB 연결과 함께 추가할 예정이다.

구성	역할
VS Code	코드 작성 및 프로젝트 관리
TypeScript	웹 POS의 주 프로그래밍 언어
React	POS 화면/UI 구성
Next.js	React 기반 웹 애플리케이션 및 서버 기능
Tailwind CSS	화면 디자인 및 반응형 UI
ESLint	코드 검사
Drizzle (다음 단계)	TypeScript에서 MariaDB를 타입 안전하게 다루는 DB 계층
MariaDB 10.11	실제 POS 데이터 저장
TDK_POS	새 POS 데이터베이스 이름

8. 다음 개발 단계

프로젝트 생성 다음 순서는 tdk - pos 폴더로 이동하여 개발 서버를 실행하고, 브라우저에서 Next.js 기본 화면이 정상적으로 뜨는지 확인하는 것이다.

```
cd tdk-pos  
npm run dev
```

그 다음 Drizzle과 MariaDB 드라이버를 설치하고 NAS의 TDK_POS 데이터베이스에 연결한다. 첫 번째 연결 테스트는 dining_tables 조회로 진행하고, 이후 직원 로그인 → 테이블 화면 → 세션 생성 → 메뉴 조회 → 주문 저장 순서로 확장한다.

9. 핵심 기억 사항

- TDK_POS = MariaDB 데이터베이스 이름
- tdk - pos = Next.js 웹 프로그램 폴더/프로젝트 이름
- 01_TDK_Web = 프로젝트 전체 작업 폴더
- docs = DB 설계 PDF와 SQL 참고자료 보관
- 실제 소스 코드는 tdk - pos/src 아래에서 작성
- PC, 태블릿, 스마트폰은 별도 프로그램이 아니라 반응형 UI를 기본으로 구성